Analysis and Implementation of a Real-Time Demonstrator on the RTIC Runtime

Gianluca Bresolin
gianluca.bresolin@studenti.unipd.it
Università degli Studi di Padova
Padova, Veneto, Italy

Tommaso Prandin
tommaso.prandin@studenti.unipd.it
Università degli Studi di Padova
Padova, Veneto, Italy

Abstract

This paper presents the implementation and analysis of a real-time demonstrator originally developed in *Ada* using the *Ravenscar Profile*, re-implemented in *Rust* using the *RTIC* framework. The demonstrator is designed to run on an *STM32F4* microcontroller and uses a *Fixed-Priority Scheduling* policy combined with *Stack Resource Policy (SRP)* for resource sharing. The analysis is performed using the *MAST* tool to verify timing properties and analyze system performance under various workloads. The results demonstrate the feasibility of adopting *Rust* and *RTIC* for real-time applications, while also highlighting differences and considerations compared to the original *Ada* implementation.

CCS Concepts

• **Software and its engineering → Real-time systems software**; **Embedded software**; *Software architectures.*

Keywords

Real-Time Kernels, Real-Time Systems, Embedded Software, Software Architectures

ACM Reference Format:
Gianluca Bresolin and Tommaso Prandin. 2025. Analysis and Implementation of a Real-Time Demonstrator on the RTIC Runtime. In *Proceedings of Real-Time Kernels and Systems Exam (RTKS '25).* ACM, New York, NY, USA, 3 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Real-time embedded systems demand precise control over timing, concurrency and resource management. Such systems are often used in critical applications where failure to meet timing constraints can lead to severe consequences. Hence, developing a real-time system requires careful consideration of the underlying runtime environment, necessitating a clear understanding of how the system will behave in order to provide sufficient guarantees on meeting timing requirements.

The choice of the runtime environment is a crucial aspect in the design of a real-time system, as it can significantly impact the system's ability to meet its desired behaviour through reliable mechanisms it offers. One of the most popular languages that meets the needs of real-time systems is *Ada*, which has been widely used in the development of safety-critical systems due to its deterministic constructs that allow high analysability of the code. While *Ada* remains a robust choice, recent advances in the *Rust* programming language have introduced new opportunities for real-time systems development.

In this paper, a real-time system originally implemented using the *Ada Ravenscar Profile* is re-implemented in *Rust* using the*Real-Time Intrupput-driven Concurrency (RTIC)* framework, which is responsible for managing the scheduling of tasks through a *Fixed-Priority Scheduling* and the resource sharing in the system under *Stack Resource Policy (SRP).*

For the analysis of the system, the *MAST* tool is used to verify the timing properties and to analyze the system under different workloads, which is run on a *STM32F4* microcontroller.

The goal of this work is to demonstrate the feasibility of using Rust and RTIC for real-time systems, while also providing some considerations and comparison with the original *Ada* implementation.

The rest of the paper is organized as follows. Section 2 presents a formal description of the system model. Section 3 introduces the *RTIC* framework and its key concepts. Sections 4 and 5 discuss the measurement of execution times and the detection of deadline misses, respectively. Section 6 provides an overview of the *MAST* tool and its analysis capabilities. In Section 7, we present the *Fixed-Priority Scheduling (FPS)* analysis of the system. Finally, Section 8 concludes the paper reporting the main findings.

2 System Model

3 RTIC

4 Execution Times

5 Deadeline Miss Detection

6 MAST

7 FPS Analysis

In this section we present our *Fixed-Priority Scheduling (FPS)* analysis of the system. The analysis was performed by modeling the system through a *MAST* model, which was then analyzed using the *MAST* tool to verify the timing properties and to analyze the system under different workloads.

The priority assignment was done according to the *Deadline Monotonic* policy, which assigns higher priorities to tasks with shorter deadlines. This choice comes from the fact that tasks in the system do not have implicit deadlines and, as theory proves, *Deadline Monotonic* performs better than *Rate Monotonic* in such cases, as it can provide feasible schedules where *Rate Monotonic* fails.

RTKS '25, Padova, Italy

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

The evaluation of the system under different utilization levels was facilitated by the use of the *Small Whetstone* algorithm to simulate computational workloads. Thanks to its configurable load and deterministic behaviour without involving the use of shared resources, it provides a straightforward way to test the system under differen utilizations by simply scaling the WCET in the different `small_whetstone` operations that are present in the *MAST* model, without the need for new execution-time measurements.

7.1 Holistic Analysis

We first conducted an holistic analysis, considering all the tasks as intendepent. As a consequence, this analysis is pessimistic, since it does not take into account the relationships between tasks. System feasibility is therefore evaluated based on the worst-case scenario at the critical instant, i.e., when all tasks are released simultaneously (not a realistic and possible case). The analysis was performed using the *Classic RM Analysis* tool in *MAST*, which implements the classic exact response time analysis for single-processor systems under *Fixed-Priority Scheduling* (although it's called *Rate Monotonic*, it bases *Deadline Monotonic* analysis).
The first results of the analysis were made considering the system under the loads provided in the original *Guided Extended Example*, i.e., with the following Whetstone workloads (accompained by their corresponding WCETs expressed in seconds).

Table 1: Whetstone Operations

Task	Workload	Rust WCET	Ada WCET
`Regular_Producer`	756	0.023639364	0.05542236
`On_Call_Producer`	278	0.008692782	0.02038018
`Activation_Log_Reader`	139	0.004346391	0.01019009

The obtained results are reported in Table 2 and Table 6. Times are expressed in seconds and we omitted the watchdogs' transactions (which were instead included in the Rust model) since they are not relevant for identifying the factors for increasing workloads.

Table 2: Holistic Analysis Results Rust

Transaction	R_{max}	Slack	Jitter	MBT
`rp_transaction`	0.024728	2006.3%	0.001066	3.030E-06
`ocp_transaction`	0.032451	8791.0%	0.023741	2.730E-06
`alr_transaction`	0.036822	21927.3%	0.032455	0.000
`ees_transaction`	4.304E-05	>=100000.0%	2.955E-05	2.730E-06
`System Utilization`	2.79%			
`System Slack`	1994.1%			

Based on these results, since the smallest slack available was 2006.3% for the `rp_transaction`, we increased the workloads with a factor of 20 to reduce it significantly. New results are reported in table 3.

Table 3: Holistic Analysis Results Rust x20

Transaction	R_{max}	Slack	Jitter	MBT
`rp_transaction`	0.474326	5.08%	0.001516	3.030E-06
`ocp_transaction`	0.647377	87.50%	0.473504	2.730E-06
`alr_transaction`	0.734413	303.91%	0.647464	0.000
`ees_transaction`	4.304E-05	>=100000.0%	2.955E-05	2.730E-06
`System Utilization`	53.76%			
`System Slack`	5.75%			

We can notice how the blocking times has not changed, since protected operations are the same as before. While we have achieved a small slack for the `rp_transaction` (5.08%), the other two transactions that involve Whetstone computations still have large slacks available. Based on these results, especially on the smallest slack of 87.50%, we decided to increase the `On_Call_Producer` and `Activation_Log_Reader` workloads by a x1.8 factor, while keeping the `Regular_Producer` workload fixed. New results are reported in table 4.

Table 4: Holistic Analysis Results Rust x1.8

Transaction	R_{max}	Slack	Jitter	MBT
`rp_transaction`	0.474326	2.73%	0.001516	3.030E-06
`ocp_transaction`	0.786600	3.91%	0.473643	2.730E-06
`alr_transaction`	0.943248	35.55%	0.786757	0.000
`ees_transaction`	4.304E-05	99263.3%	2.955E-05	2.730E-06
`System Utilization`	58.86%			
`System Slack`	1.98%			

Finally, based on the last useful slack available of 35.55% for the `alr_transaction`, we increased its workload by a x1.35 factor. New results are reported in table 5.

Table 5: Holistic Analysis Results Rust x1.35

Transaction	R_{max}	Slack	Jitter	MBT
`rp_transaction`	0.474326	0.00%	0.001516	3.030E-06
`ocp_transaction`	0.786600	0.00%	0.473643	2.730E-06
`alr_transaction`	0.998068	0.39%	0.786812	0.000
`ees_transaction`	4.304E-05	99263.3%	2.955E-05	2.730E-06
`System Utilization`	60.68%			
`System Slack`	0.39%			

Even though the `ees_transaction` has still a large slack available, since it does not perform any Whetstone computation, it is not possible to increase the utilization of the system any further. Besides that, even an eventual increase of the execution time of `ees_transaction` would not affect significantly the utilization of the system. In conclusion, under an holistic analysis, we achieved an utilization of the system equal to 60.68%.

Table 6: Holistic Analysis Results Ada

Transaction	R_{max}	Slack	Jitter	MBT
rp_transaction	0.056703	796.09%	0.001243	1.361E-06
ocp_transaction	0.076194	3530.5%	0.055792	1.267E-06
alr_transaction	0.086455	8867.2%	0.076238	0.000
ees_transaction	1.882E-05	>=100000.0%	1.141E-05	1.267E-06
System Utilization	6.69%			
System Slack	778.52%			

Based on these results, since the smallest slack available was 796.09% for the rp_transaction, we increased the workloads with a factor of 8.5 to reduce it significantly. New results are reported in table 7.

Table 7: Holistic Analysis Results Ada x8.5

Transaction	R_{max}	Slack	Jitter	MBT
rp_transaction	0.474027	5.47%	0.002900	1.361E-06
ocp_transaction	0.646972	87.89%	0.473719	1.267E-06
alr_transaction	0.733964	304.30%	0.647322	0.000
ees_transaction	1.882E-05	>=100000.0%	1.141E-05	1.267E-06
System Utilization	53.86%			
System Slack	5.75%			

Again, we can notice how the blocking times has not changed, since protected operations are the same as before. While we have achieved a small slack for the rp_transaction (5.47%), the other two transactions that involve Whetstone computations still have large slacks available. Based on these results, especially on the smallest slack of 87.89%, we decided to increase the On_Call_Producer and
Activation_Log_Reader workloads by a x1.8 factor, while keeping the Regular_Producer workload fixed. New results are reported in table 8.

Table 8: Holistic Analysis Results Ada x1.8

Transaction	R_{max}	Slack	Jitter	MBT
rp_transaction	0.474027	2.73%	0.002900	1.361E-06
ocp_transaction	0.786113	4.30%	0.474274	0.000
alr_transaction	0.942671	35.94%	0.786736	1.361E-06
ees_transaction	1.882E-05	>=100000.0%	1.141E-05	1.267E-06
System Utilization	58.94%			
System Slack	1.98%			

Finally, based on the last useful slack available of 35.94% for the alr_transaction, we increased its workload by a x1.35 factor. New results are reported in table 9.

Table 9: Holistic Analysis Results Ada x1.35

Transaction	R_{max}	Slack	Jitter	MBT
rp_transaction	0.474027	0.00%	0.002900	1.361E-06
ocp_transaction	0.786113	0.39%	0.474274	0.000
alr_transaction	0.997457	0.39%	0.786954	1.361E-06
ees_transaction	1.882E-05	20788.3%	1.141E-05	1.267E-06
System Utilization	60.76%			
System Slack	0.39%			

Again, the considerations made before upon the ees_transaction apply here. In conclusion, under an holistic analysis, we achieved an utilization of the system equal to 60.76%.

7.2 Phased Analysis

8 Conclusions

9 Citations and Bibliographies

References